ISGA Rabat | École d'Ingénieurs - École de Management

2ème année du Cycle Ingénieur - Ingénierie des Systèmes Informatiques

Projet de Fin d'Année (PFA)

THÈME

Système de génération d'avatars parlants pour l'interaction pédagogique

Nom du projet : EduAvatar

Réalisé par :

AIT ALI Abdessamad

KAJJA Hamza

Encadré par :

Youness TABII

Année universitaire 2025 / 2026

Remerciements

Au terme de ce projet, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce travail.

Nous remercions en premier lieu notre encadrant, Youness TABII, pour sa disponibilité, ses conseils avisés et le suivi rigoureux qu'il nous a accordé tout au long de la réalisation de ce projet.

Nous adressons également nos remerciements à l'ensemble du corps enseignant de ISGA pour la qualité de la formation dont nous avons bénéficié, ainsi qu'au jury qui nous fait l'honneur d'évaluer ce travail.

Nous remercions enfin nos familles et nos proches pour leur soutien constant et leurs encouragements durant tout notre parcours universitaire.

Résumé

Ce rapport présente la conception et la réalisation d'EduAvatar, une plateforme web pédagogique dont l'objectif est de transformer des contenus de cours classiques en interactions vivantes grâce à des avatars parlants. Le système permet à un apprenant de consulter des cours, d'écouter un professeur virtuel expliquer une leçon, de lui poser des questions à l'écrit ou à la voix, et de visualiser un avatar animé prononçant les réponses.

La solution s'appuie sur une architecture trois-tiers : un frontend Angular, un backend Node.js/Express exposant une API REST sécurisée par JWT, et une base de données PostgreSQL. La dimension intelligence artificielle repose exclusivement sur des services gratuits : un grand modèle de langage (Groq / LLaMA) pour la génération des explications, Whisper pour la reconnaissance vocale, Microsoft Edge TTS pour la synthèse vocale, et SadTalker exécuté sur Google Colab pour l'animation faciale synchronisée sur l'audio.

Mots-clés : avatar parlant, e-learning, intelligence artificielle, TTS, reconnaissance vocale, LLM, SadTalker, Angular, Node.js, PostgreSQL.

Abstract

This report presents the design and implementation of EduAvatar, an educational web platform whose goal is to turn conventional course content into lively interactions using talking avatars. The system lets a learner browse courses, listen to a virtual teacher explain a lesson, ask questions in writing or by voice, and watch an animated avatar pronouncing the answers.

The solution relies on a three-tier architecture: an Angular frontend, a Node.js/Express backend exposing a JWT-secured REST API, and a PostgreSQL database. The artificial intelligence layer is built exclusively on free services: a large language model (Groq / LLaMA) for explanations, Whisper for speech recognition, Microsoft Edge TTS for speech synthesis, and SadTalker running on Google Colab for audio-driven face animation.

Keywords: talking avatar, e-learning, artificial intelligence, TTS, speech recognition, LLM, SadTalker, Angular, Node.js, PostgreSQL.

Table des matières

Remerciements 2
Résumé 3
Abstract 3
Liste des figures 6
Liste des tableaux 7
Introduction générale 8
Chapitre 1. Contexte, problématique et état de l'art 9
1.1. Contexte du projet 9
1.2. Problématique 9
1.3. Objectifs 9
1.4. État de l'art 9
1.4.1. Les avatars parlants 9
1.4.2. La synthèse vocale (TTS) 10
1.4.3. La reconnaissance vocale (ASR) 10
1.4.4. Les grands modèles de langage (LLM) 10
1.4.5. Solutions existantes et positionnement 10
Chapitre 2. Analyse et spécification des besoins 11
2.1. Acteurs du système 11
2.2. Besoins fonctionnels 11
2.3. Besoins non fonctionnels 11
2.4. Cas d'utilisation principaux 12
2.5. Environnement et technologies 12
Chapitre 3. Conception de la solution 14
3.1. Architecture globale 14
3.2. Architecture du backend 14
3.3. Architecture du frontend 15
3.4. Modèle de données 15
3.5. Sécurité 16
3.6. API REST 17
3.7. Pipeline d'intelligence artificielle 18
3.7.1. Scénario « question vocale → réponse parlée » 18

3.7.2. Scénario « génération d'avatar parlant » 19
3.8. Déploiement 20
Chapitre 4. Réalisation 22
4.1. Environnement de développement 22
4.2. Initialisation du backend 22
4.3. Accès aux données 22
4.4. Authentification et sécurité 23
4.5. Le service LLM (Groq) 23
4.6. La reconnaissance vocale (Whisper) 23
4.7. La synthèse vocale (Edge TTS) 23
4.8. La génération de l'avatar parlant (SadTalker) 24
4.9. Le frontend Angular 24
4.10. Conteneurisation et déploiement 24
4.11. Aperçu des interfaces 25
Chapitre 5. Tests et validation 31
5.1. Stratégie de test 31
5.2. Scénarios testés 31
5.3. Tests de robustesse 31
5.4. Performances observées 31
Chapitre 6. Bilan, difficultés et perspectives 33
6.1. Bilan 33
6.2. Difficultés rencontrées 33
6.3. Perspectives d'amélioration 33
Conclusion générale 34
Webographie et références 35
Annexes 36
Annexe A. Variables d'environnement 36
Annexe B. Comptes de démonstration (seed) 36
Annexe C. Structure du projet 36

Liste des figures

FIGURE 1 — DIAGRAMME DE CAS D'UTILISATION 12
FIGURE 2 — ARCHITECTURE EN COMPOSANTS 14
FIGURE 3 — DIAGRAMME DE CLASSES 16
FIGURE 4 — DIAGRAMME D'ACTIVITE : AUTHENTIFICATION 17
FIGURE 5 — DIAGRAMME DE SEQUENCE : QUESTION VOCALE ET REPONSE PARLEE 19
FIGURE 6 — DIAGRAMME DE SEQUENCE : GENERATION D'UN AVATAR PARLANT 20
FIGURE 7 — DIAGRAMME DE DÉPLOIEMENT 20
FIGURE 8 — PAGE DE CONNEXION 25
FIGURE 9 — TABLEAU DE BORD DE L'APPRENANT 25
FIGURE 10 — LISTE DES COURS (APPRENANT) 26
FIGURE 11 — DETAIL D'UN COURS ET DE SES LEÇONS 26
FIGURE 12 — EXPLICATION D'UNE LEÇON PAR L'IA 27
FIGURE 13 — CHAT AVEC L'AVATAR PEDAGOGIQUE 27
FIGURE 14 — TABLEAU DE BORD ADMINISTRATEUR 28
FIGURE 15 — AJOUT D'UN NOUVEAU COURS 28
FIGURE 16 — GESTION DES UTILISATEURS 29
FIGURE 17 — PAGE DES STATISTIQUES 29
FIGURE 18 — CREATION ET GENERATION D'UN AVATAR PARLANT 30

Liste des tableaux

TABLE 1 — SOLUTIONS EXISTANTES ET POSITIONNEMENT 10
TABLE 2 — ACTEURS DU SYSTEME 11
TABLE 3 — BESOINS FONCTIONNELS 11
TABLE 4 — CAS D'UTILISATION PRINCIPAUX 12
TABLE 5 — ENVIRONNEMENT ET TECHNOLOGIES 13
TABLE 6 — ARCHITECTURE DU FRONTEND 15
TABLE 7 — MODELE DE DONNEES 15
TABLE 8 — API REST 18
TABLE 9 — ENVIRONNEMENT DE DÉVELOPPEMENT 22
TABLE 10 — SCÉNARIOS TESTÉS 31
TABLE 11 — PERFORMANCES OBSERVÉES 32
TABLE 12 — VARIABLES D'ENVIRONNEMENT 36
TABLE 13 — COMPTES DE DÉMONSTRATION 36

Introduction générale

L'enseignement assisté par ordinateur a connu, ces dernières années, une mutation profonde portée par les progrès de l'intelligence artificielle générative. Les supports pédagogiques traditionnels — polycopiés, diaporamas, vidéos statiques — peinent à maintenir l'attention des apprenants et offrent peu d'interactivité. Parallèlement, les modèles de langage, la synthèse vocale neuronale et l'animation faciale sont devenus accessibles via des interfaces simples et, pour beaucoup, gratuites.

C'est dans ce contexte que s'inscrit le projet EduAvatar, dont le thème est la « génération d'avatars parlants pour l'interaction pédagogique ». L'ambition est de concevoir une plateforme web capable de donner un visage et une voix à un professeur virtuel qui explique, répond et interagit avec l'apprenant de manière naturelle.

L'objectif général se décline en plusieurs objectifs spécifiques : mettre en place une gestion complète des cours et des leçons ; permettre à l'utilisateur de dialoguer avec un assistant pédagogique intelligent ; offrir une reconnaissance vocale transformant la parole en texte ; produire une synthèse vocale de qualité ; et générer une vidéo d'avatar parlant synchronisée sur l'audio.

Ce rapport est organisé en six chapitres. Le premier présente le contexte, la problématique et l'état de l'art. Le deuxième détaille l'analyse et la spécification des besoins. Le troisième expose la conception et l'architecture. Le quatrième décrit la réalisation. Le cinquième traite des tests et de la validation. Le sixième dresse le bilan, les difficultés rencontrées et les perspectives. Une conclusion générale clôt le document.

Chapitre 1. Contexte, problématique et état de l'art

1.1. Contexte du projet

Le projet s'inscrit dans le domaine des technologies éducatives (EdTech) et de l'intelligence artificielle appliquée à l'éducation. Les plateformes d'apprentissage en ligne se sont largement démocratisées, mais la plupart restent passives : l'apprenant lit ou regarde, sans véritable échange. Les avatars parlants offrent un canal d'interaction plus humain, capable de renforcer l'engagement et la mémorisation.

EduAvatar vise à démontrer qu'il est possible de construire une telle plateforme en s'appuyant sur des briques gratuites ou open source, sans infrastructure GPU payante : Groq pour l'inférence du modèle de langage, des API hébergées pour la transcription et la synthèse vocale, et Google Colab pour l'animation de l'avatar.

1.2. Problématique

La problématique centrale peut se formuler ainsi :

« Comment concevoir et réaliser une plateforme pédagogique interactive capable de générer des avatars parlants à partir de contenus de cours, tout en restant accessible et gratuite, et en offrant une expérience conversationnelle naturelle à l'apprenant ? »

Cette question en soulève plusieurs autres : comment garantir la qualité et la fluidité des réponses, comment synchroniser la voix et l'animation labiale, comment sécuriser l'accès aux ressources, et comment rester robuste face aux limites des services externes gratuits (quotas, disponibilité, latence) ?

1.3. Objectifs

Le système poursuit les objectifs suivants :

- Proposer une gestion complète des cours et des leçons (création, consultation, suivi).
- Fournir un assistant pédagogique intellige capable d'expliquer une leçon et de répondre aux questions.
- Intégrer la reconnaissance vocale pour permettre à l'utilisateur de parler au lieu de taper.
- Générer la voix de l'avatar (synthèse vocale) et une animation faciale synchronisée.
- Sécuriser l'application (authentification, rôles, protection des routes et des fichiers).
- Déployer l'ensemble de manière reproductible via Docker.

1.4. État de l'art

1.4.1. Les avatars parlants

Un avatar parlant (talking head) est un personnage virtuel dont le visage est animé de manière cohérente avec un signal audio. On distingue trois grandes familles d'approches : les modèles 3D paramétriques (par exemple des rigs de type blend shapes), les approches 2D basées sur des maillages de déformation (warpping) et, plus récemment, les modèles génératifs neuronaux. SadTalker, utilisé dans ce projet,

appartient à la famille des approches 2D par apprentissage profond : à partir d'une photo et d'un audio, il prédit des coefficients 3D de la tête et des expressions, puis génère une vidéo photoréaliste.

1.4.2. La synthèse vocale (TTS)

La synthèse vocale a connu un saut qualitatif avec les modèles neuronaux (Tacotron, FastSpeech, VITS…). Aujourd'hui, des solutions comme Microsoft Edge TTS proposent des voix neuronales naturelles, y compris en français (voix Denise, Henri, Éloïse), sans clé d'API ni coût. C'est le choix retenu pour ce projet.

1.4.3. La reconnaissance vocale (ASR)

La reconnaissance automatique de la parole repose majoritairement sur l'architecture Whisper d'OpenAI, réputée pour sa robustesse multilingue. Elle est disponible via des API gratuites (Groq, Hugging Face), ce qui permet de l'intégrer sans serveur dédié.

1.4.4. Les grands modèles de langage (LLM)

Les LLM (GPT, LLaMA, Mistral…) génèrent des réponses textuelles contextuelles. Pour l'assistant pédagogique, ils permettent de produire des explications adaptées au niveau de l'apprenant. Le projet s'appuie sur l'API Groq, qui offre une inférence très rapide sur le modèle LLaMA, avec un quota gratuit.

1.4.5. Solutions existantes et positionnement

Solution	Type	Limites	Apport d'EduAvatar
ChatGPT / Gemini	Chatbot texte	Pas d'avatar animé intégré	Avatar parlant + voix
HeyGen / Synthesia	Avatars commerciaux	Payant, fermé	Gratuit, open source
Moodle + plugins	LMS	Interactivité IA limitée	IA conversationnelle intégrée
SadTalker seul	Générateur de vidéo	Pas de plateforme pédagogique	Plateforme complète

Table 1 — Solutions existantes et positionnement

EduAvatar ne prétend pas rivaliser avec les solutions commerciales sur le plan du photoréalisme. Son originalité réside dans l'intégration, à coût nul, de toute la chaîne pédagogique et IA au sein d'une application web cohérente.

Chapitre 2. Analyse et spécification des besoins

2.1. Acteurs du système

Acteur	Description	Principaux droits
Visiteur	Utilisateur non authentifié	S'inscrire, se connecter
Apprenant (user)	Étudiant connecté	Consulter les cours, chatter, parler, générer des vidéos
Administrateur (admin)	Gestionnaire de contenu	CRUD cours/leçons/utilisateurs, statistiques globales

Table 2 — Acteurs du système

2.2. Besoins fonctionnels

Les besoins fonctionnels sont regroupés par domaine :

Réf.	Besoin	Acteur
BF1	Créer un compte et se connecter via email/mot de passe	Visiteur
BF2	Consulter la liste des cours et le détail avec leçons	Apprenant
BF3	Créer / modifier / supprimer des cours et des leçons	Admin
BF4	Créer un avatar à partir d'une photo (upload)	Apprenant
BF5	Générer la vidéo d'un avatar parlant à partir d'un texte	Apprenant
BF6	Obtenir une explication IA d'une leçon	Apprenant
BF7	Poser une question libre à l'IA	Apprenant
BF8	Dicter une question au micro (transcription)	Apprenant
BF9	Écouter la réponse (synthèse vocale) avec sous-titres	Apprenant
BF10	Chatter avec l'avatar via des conversations persistantes	Apprenant
BF11	Marquer une leçon comme terminée / suivre sa progression	Apprenant
BF12	Consulter les statistiques (dont globales pour l'admin)	Admin / Apprenant
BF13	Gérer les comptes utilisateurs	Admin

Table 3 — Besoins fonctionnels

2.3. Besoins non fonctionnels

- Sécurité : mots de passe hachés (bcrypt), authentification par JWT, routes protégées, contrôle de rôle.
- Performance : réponses IA rapides (Groq), génération TTS par phrases en parallèle.
- Ergonomie : interface responsive (Angular + Tailwind CSS), retours visuels (chargement, erreurs).
- Fiabilité : mécanismes de repli (fallback) lorsque un service IA est indisponible.
- Portabilité : conteneurisation Docker (dev et production) avec nginx.

- Maintenabilité : séparation claire entre modèles, contrôleurs, services et routes.

2.4. Cas d'utilisation principaux

Les cas d'utilisation les plus significatifs sont décrits ci-dessous.

Cas d'utilisation	Acteur	Scénario nominal
S'authentifier	Visiteur	Saisie email/mdp → vérification → émission du token JWT
Consulter un cours	Apprenant	Sélection d'un cours → affichage des leçons
Expliquer une leçon	Apprenant	Clic « Expliquer » → appel LLM → réponse audio + texte
Poser une question vocale	Apprenant	Enregistrement micro → Whisper → LLM → voix
Générer un avatar parlant	Apprenant	Texte → Edge TTS → SadTalker → vidéo MP4
Gérer le contenu	Admin	CRUD cours/leçons via formulaires dédiés

Table 4 — Cas d'utilisation principaux

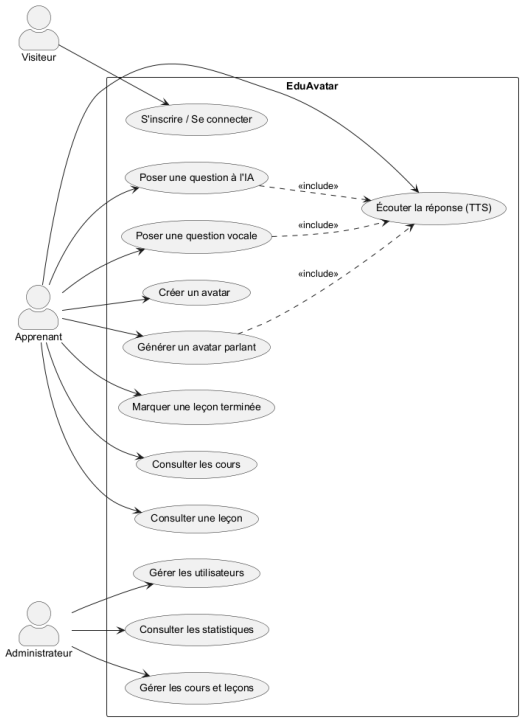


Figure 1 — Diagramme de cas d'utilisation

2.5. Environnement et technologies

Le tableau suivant récapitule le choix des technologies et leur justification.

Couche	Technologie	Justification

Frontend	Angular 17, TypeScript	Framework structuré, SPA réactive, injection de dépendances
UI	Tailwind CSS + Angular Material	Développement rapide, composants prêts à l'emploi
Backend	Node.js 20, Express	JavaScript unifié, écosystème riche, API REST simple
BDD	PostgreSQL 16	Relationnel robuste, types avancés, requêtes SQL paramétrées
Sécurité	JWT + bcryptjs	Authentification stateless, hachage des mots de passe
LLM	Groq (LLaMA)	Inférence rapide, quota gratuit, API compatible OpenAI
ASR	Whisper (Groq / Hugging Face)	Transcription multilingue de grande qualité, gratuit
TTS	Microsoft Edge TTS	Voix neuronales naturelles, gratuites, sans clé
Avatar	SadTalker (Colab + ngrok)	Animation faciale réaliste, GPU gratuit via Colab
DevOps	Docker, docker-compose, nginx	Reproductibilité, déploiement simplifié

Table 5 — Environnement et technologies

Chapitre 3. Conception de la solution

3.1. Architecture globale

EduAvatar adopte une architecture trois-tiers classique, enrichie d'une couche de services d'intelligence artificielle externes :

```
┌──────────────────────────────┐
│        Navigateur web        │
│   (Angular 17 + Tailwind)    │
└──────────────┬───────────────┘
               │ HTTP / REST (JSON, multipart)
┌──────────────▼───────────────┐
│   API Backend Node/Express   │
│  JWT • Controllers • Services│
└──┬──────────┬──────────┬─────┘
   │          │          │
┌──▼──┐  ┌────▼────┐  ┌──▼──────────────┐
│Postgres│ │ Fichiers│ │ Services IA     │
│  BDD   │ │ uploads │ │ Groq/Whisper/Edge │
└────────┘ └─────────┘ │ SadTalker (Colab) │
                       └─────────────────┘
```

En développement, le frontend Angular (port 4200) redirige les appels /api et /uploads vers le backend (port 3000) via un proxy (proxy.conf.json), ce qui évite les problèmes de CORS. En production, nginx sert le build Angular et joue le rôle de reverse proxy.

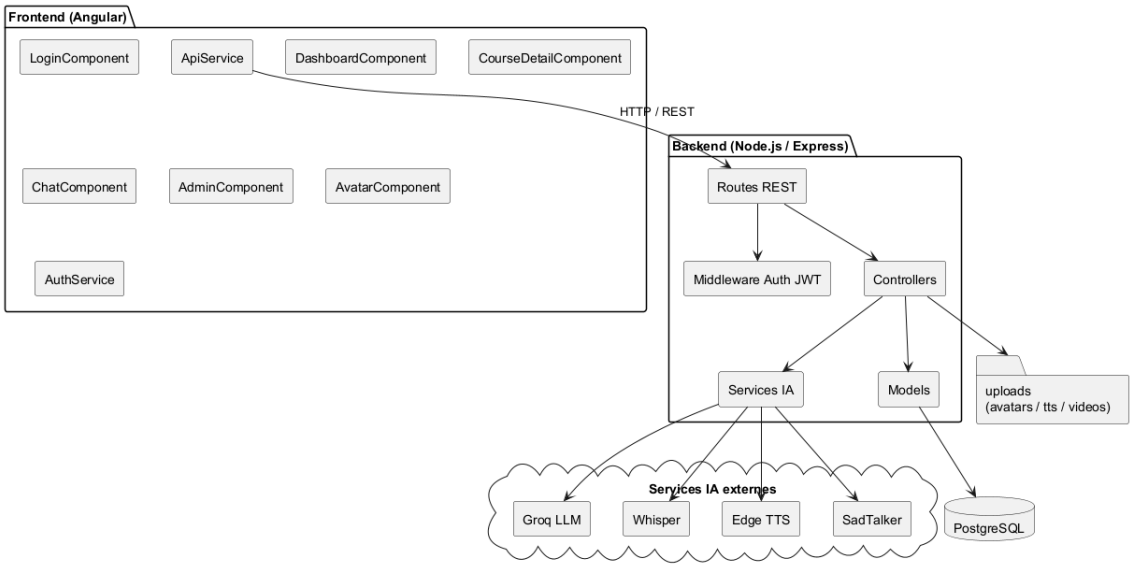


Figure 2 — Architecture en composants

3.2. Architecture du backend

Le backend suit une organisation en couches, ce qui facilite la maintenance et les tests :

- config/ : configuration de la connexion PostgreSQL (pool de connexions).

- models/ : classes d'accès aux données (User, Course, Lesson, Avatar, Conversation, Message, Progress).
- controllers/ : logique de traitement des requêtes (auth, user, course, lesson, avatar, conversation, stats, ai).
- services/ : intégration des services IA (groq, whisper, tts, sadtalker).
- middleware/ : authentification JWT, autorisation admin, gestion des uploads (Multer).
- routes/ : définition des points d'entrée de l'API REST.

3.3. Architecture du frontend

L'application Angular est organisée en composants autonomes, en services et en guards :

Élément	Rôle
LoginComponent	Authentification de l'utilisateur
DashboardComponent	Tableau de bord, cours et progression
CourseDetailComponent	Détail d'un cours, explication IA, questions vocales, lecture audio
ChatComponent	Conversation textuelle et vocale avec l'avatar
AdminComponent	Gestion des cours, leçons et utilisateurs
AvatarComponent	Création d'avatar et génération de vidéo parlante
AuthService	Gestion du token JWT et de l'état de connexion
ApiService / *Service	Communication avec l'API REST
AuthGuard / AdminGuard	Protection des routes privées et administrateur

Table 6 — Architecture du frontend

3.4. Modèle de données

La base de données relationnelle comporte les tables principales suivantes :

Table	Rôle	Champs clés
users	Comptes utilisateurs	id, username, email, password (hash), role, picture, created_at
courses	Cours	id, title, description, level, category, created_by
lessons	Leçons d'un cours	id, course_id, title, content, order_index
avatars	Avatars utilisateurs	id, user_id, name, source_image, video_url
conversations	Sessions de chat	id, user_id, avatar_id, course_id, created_at
messages	Messages échangés	id, conversation_id, role, content, created_at
progress	Progression de l'apprenant	id, user_id, lesson_id, completed, updated_at

Table 7 — Modèle de données

Les relations principales sont : un utilisateur possède plusieurs avatars, conversations et entrées de progression ; un cours contient plusieurs leçons ; une conversation contient plusieurs messages et peut être liée à un avatar et à un cours.

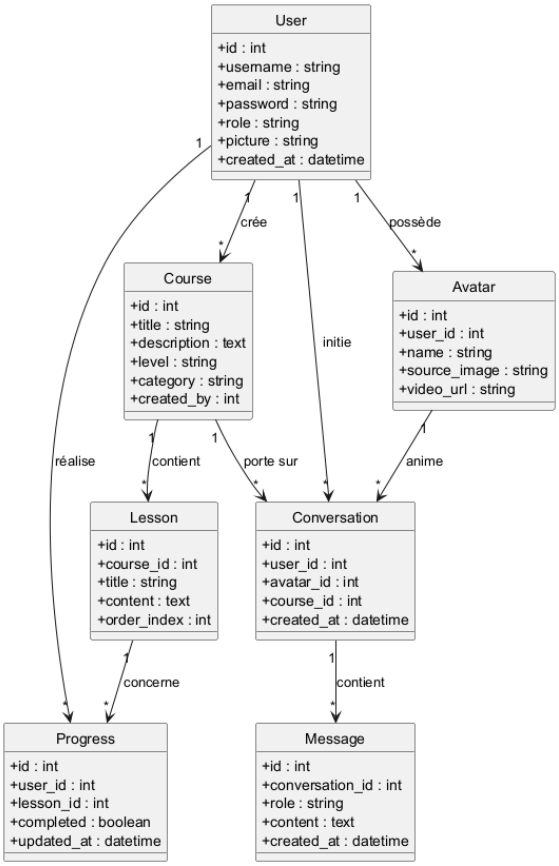


Figure 3 — Diagramme de classes

3.5. Sécurité

- Hachage des mots de passe avec bcryptjs (10 tours de salage).
- Jetons JWT signés avec un secret (JWT_SECRET) et vérifiés à chaque requête protégée.
- Middleware authenticate : vérifie le header Authorization « Bearer <token> ».
- Middleware authorizeAdmin : restreint certaines routes au rôle admin.
- Requêtes SQL paramétrées (protection contre les injections).
- Contrôle des types MIME et de la taille (10 Mo) lors des uploads.

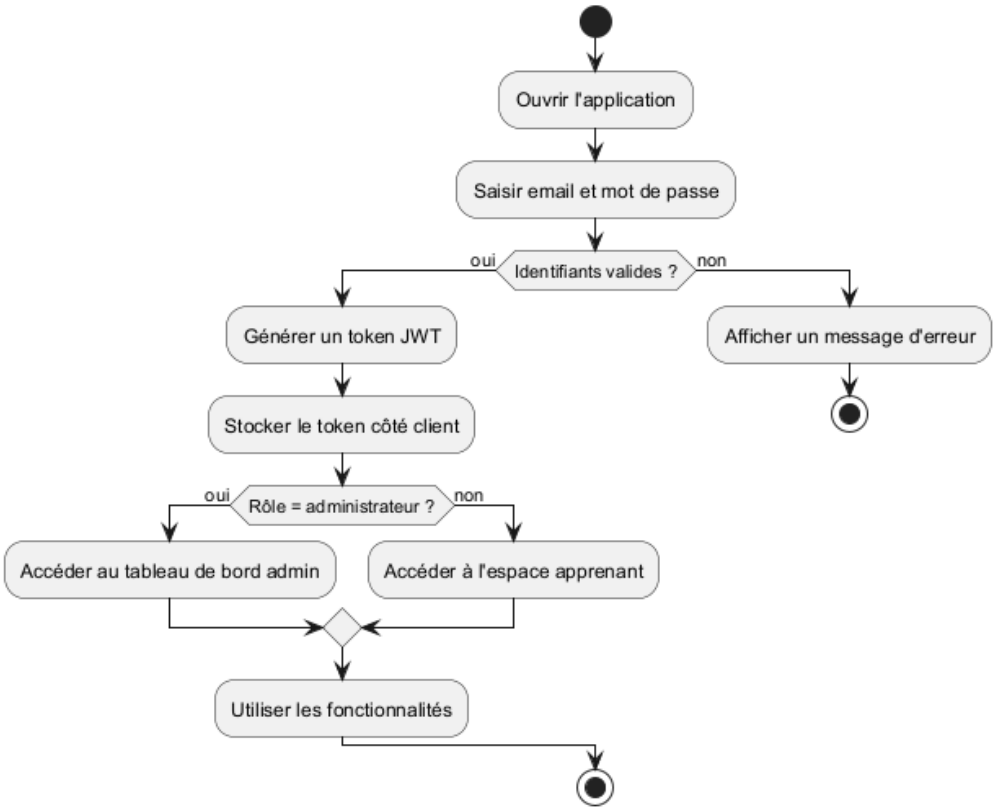


Figure 4 — Diagramme d'activité : authentification

3.6. API REST

L'API expose les principaux points d'entrée suivants :

Méthode	Route	Description	Accès
POST	/api/auth/register	Inscription	Public
POST	/api/auth/login	Connexion (retourne le JWT)	Public
GET	/api/auth/me	Profil courant	Authentifié
GET	/api/courses	Liste des cours	Authentifié
GET	/api/courses/:id	Détail d'un cours + leçons	Authentifié
POST	/api/courses	Créer un cours	Admin
PUT/DELETE	/api/courses/:id	Modifier / supprimer un cours	Admin
POST	/api/courses/:id/lessons	Créer une leçon	Admin
POST	/api/lessons/:id/complete	Marquer une leçon terminée	Authentifié
POST	/api/avatars	Créer un avatar (upload photo)	Authentifié

POST	/api/avatars/:id/generate	Générer la vidéo parlante	Authentifié
POST	/api/conversations	Créer une conversation	Authentifié
POST	/api/conversations/:id/messages	Envoyer un message (LLM)	Authentifié
POST	/api/ai/explain	Expliquer une leçon	Authentifié
POST	/api/ai/ask	Poser une question à l'IA	Authentifié
POST	/api/ai/speak	Synthèse vocale simple	Authentifié
POST	/api/ai/speak-sentences	Synthèse vocale par phrase	Authentifié
POST	/api/ai/transcribe	Transcription audio (Whisper)	Authentifié
GET	/api/stats	Statistiques globales	Admin
GET	/api/stats/user	Statistiques de l'utilisateur	Authentifié
GET	/api/health	Vérification de l'état du service	Public

Table 8 — API REST

3.7. Pipeline d'intelligence artificielle

La chaîne de traitement IA est le cœur du projet. Elle peut être décrite en deux scénarios.

3.7.1. Scénario « question vocale → réponse parlée »

```
Micro (navigateur)
   │  audio/webm
   ▼
POST /api/ai/transcribe  ──► Whisper (Groq, repli Hugging Face)
   │  texte
   ▼
POST /api/ai/ask         ──► LLM Groq (LLaMA)  → réponse pédagogique
   │  texte
   ▼
POST /api/ai/speak-sentences ──► Edge TTS → un MP3 par phrase
   │  URLs audio + phrases
   ▼
Lecture séquentielle côté client avec sous-titres synchronisés
```

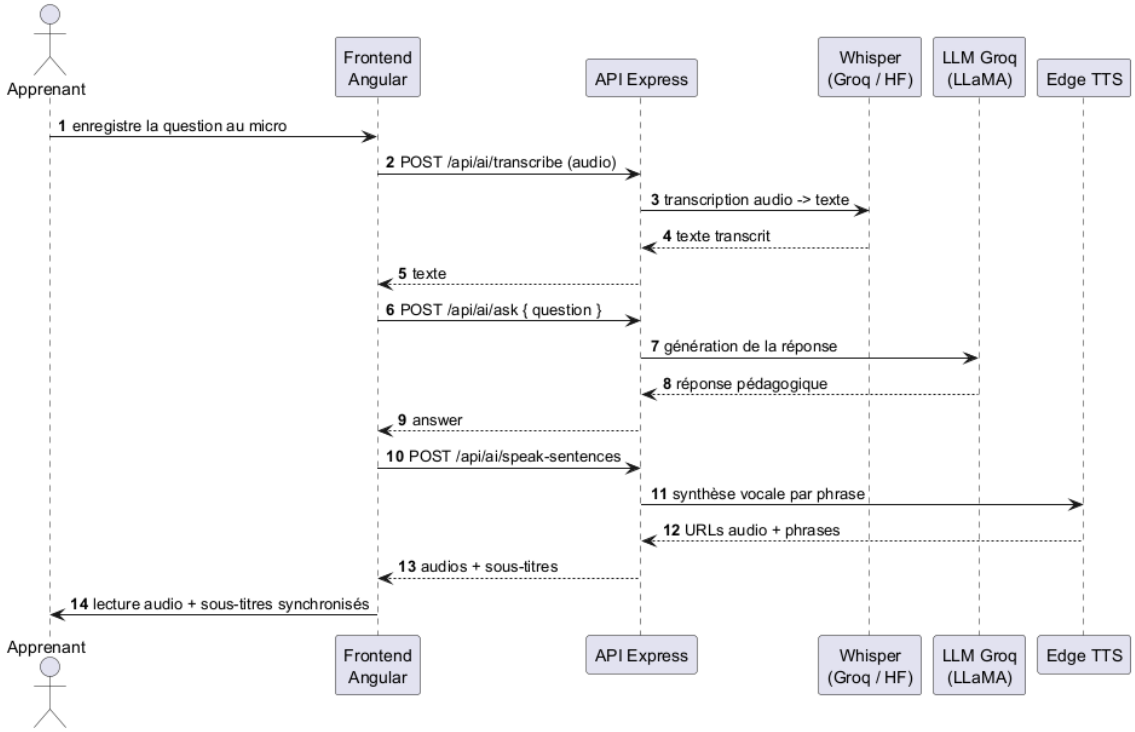


Figure 5 — Diagramme de séquence : question vocale et réponse parlée

3.7.2. Scénario « génération d'avatar parlant »

```
Texte de la réponse
   │
   ▼
Edge TTS (gratuit) ──► fichier audio MP3
   │
   ▼
SadTalker (notebook Colab via ngrok)
   │  POST /generate (image + audio)
   ▼
Vidéo MP4 sauvegardée dans /uploads/videos → lue par le navigateur
```

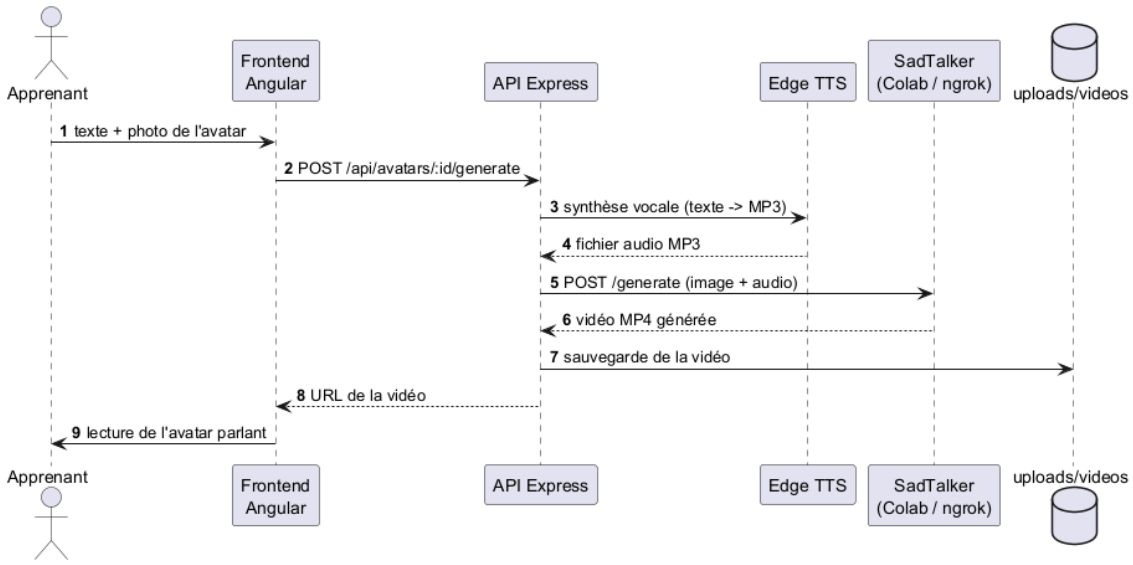


Figure 6 — Diagramme de séquence : génération d'un avatar parlant

Remarque : la génération SadTalker s'exécute sur un GPU distant (Google Colab) et peut prendre plusieurs minutes ; le délai d'attente du service est fixé en conséquence à 10 minutes. En cas d'indisponibilité, la plateforme reste utilisable grâce au repli sur la synthèse vocale et le texte.

3.8. Déploiement

Le diagramme de déploiement ci-dessous montre la répartition des composants entre le poste client, le serveur hôte (conteneurs Docker) et les services d'intelligence artificielle externes :

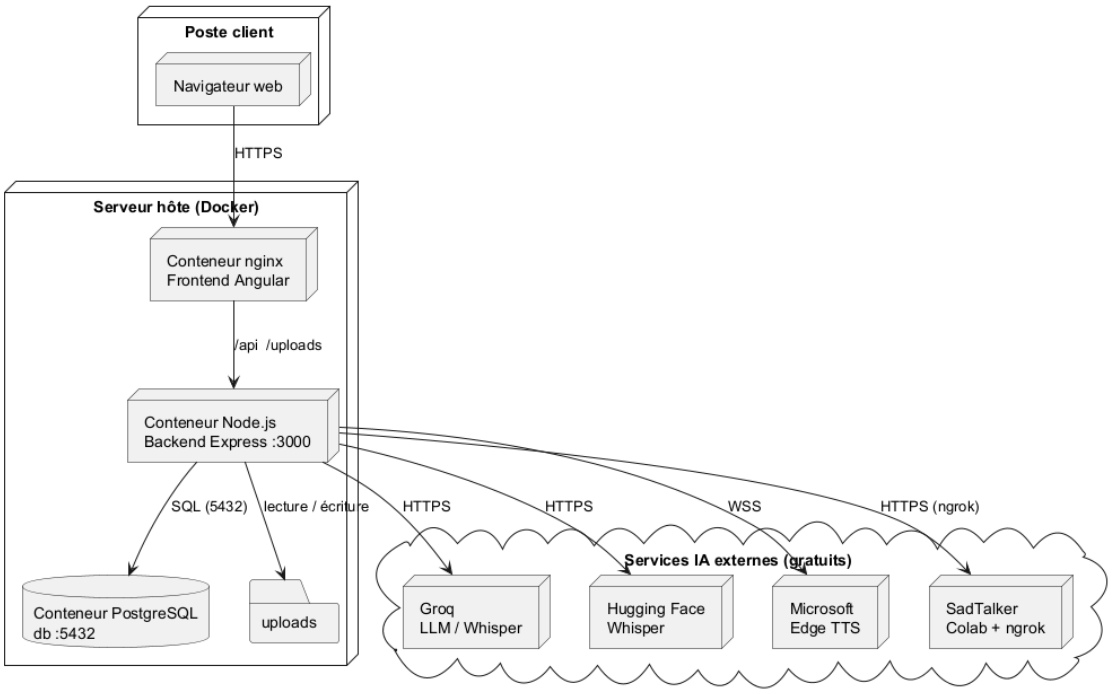


Figure 7 — Diagramme de déploiement

Chapitre 4. Réalisation

4.1. Environnement de développement

Outil	Version / usage
Node.js	20 LTS — exécution du backend et build Angular
Angular CLI	17 — génération et build du frontend
PostgreSQL	16 (image Docker postgres:16-alpine)
Docker / docker-compose	Orchestration des trois services
Visual Studio Code	Édition du code
Git	Gestion de versions

Table 9 — Environnement de développement

4.2. Initialisation du backend

Le point d'entrée app.js configure Express, le CORS, le parsing JSON, les fichiers statiques, les routes API et la gestion centralisée des erreurs, puis initialise la base et démarre le serveur :

```
const app = express();
app.use(cors({ origin: process.env.CORS_ORIGIN, credentials: true }));
app.use(express.json({ limit: '10mb' }));
app.use('/uploads', express.static(path.join(__dirname, '..', 'uploads')));
app.use('/api/auth', authRoutes);
app.use('/api/courses', courseRoutes);
// ... autres routes
async function startServer() {
  await initDatabase();
  app.listen(PORT, () => console.log('EduAvatar Backend démarré'));
}
```

4.3. Accès aux données

Les modèles encapsulent les requêtes SQL paramétrées. Exemple de création d'utilisateur avec hachage du mot de passe :

```
static async create({ username, email, password, role = 'user' }) {
  const hashedPassword = await bcrypt.hash(password, 10);
  const { rows } = await pool.query(
    `INSERT INTO users (username, email, password, role)
     VALUES ($1, $2, $3, $4)
     RETURNING id, username, email, role, created_at`,
    [username, email, hashedPassword, role]
  );
  return rows[0];
}
```

4.4. Authentification et sécurité

Le middleware d'authentification vérifie le jeton JWT présent dans l'en-tête Authorization, charge l'utilisateur correspondant et l'ajoute à la requête. Un second middleware contrôle le rôle administrateur :

```
const token = req.headers.authorization.split(' ')[1];
const decoded = jwt.verify(token, process.env.JWT_SECRET);
const user = await User.findById(decoded.id);
if (!user) return res.status(401).json({ error: 'Utilisateur introuvable' });
req.user = user;
next();
```

4.5. Le service LLM (Groq)

Le service groq.service.js construit un « system prompt » qui définit la personnalité du professeur virtuel. Les consignes insistent sur le français oral, la concision (le texte sera lu à voix haute) et l'usage d'exemples concrets :

```
systemPrompt = `Tu es ${avatarName}, un professeur passionné ...
  Tu parles TOUJOURS EN FRANÇAIS ...
  Utilise des mots simples et des EXEMPLES du quotidien ...
  IMPORTANT : reste CONCIS car ton discours sera LU À VOIX HAUTE.`

body: {
  model: this.groqModel,        // llama-3.3-70b-versatile
  messages, temperature: 0.5,
  max_tokens: 300, top_p: 0.9
}
```

Un mécanisme de repli (fallbackResponse) garantit qu'une réponse est toujours renvoyée, même si l'API externe échoue.

4.6. La reconnaissance vocale (Whisper)

Le service whisper.service.js implémente une stratégie de repli à deux niveaux : il tente d'abord Groq (rapide), puis Hugging Face. L'audio est transmis en multipart/form-data, et la langue est forcée à « fr » pour améliorer la précision :

```
async transcribe(audioBuffer, originalName) {
  if (this.groqKey) { try { return await this.transcribeGroq(...); } catch {} }
  if (this.hfKey)  { try { return await this.transcribeHuggingFace(...); } catch {} }
  throw new Error('[Whisper] Tous les fournisseurs ont échoué');
}
```

4.7. La synthèse vocale (Edge TTS)

Le service tts.service.js exploite le paquet msedge-tts, qui reproduit le protocole WebSocket de Microsoft Edge, pour produire des MP3 de voix neuronales françaises sans clé d'API. Une variante « par phrases » découpe le texte et génère un fichier audio par phrase en parallèle (concurrence limitée à 3), ce qui permet une lecture séquentielle avec des sous-titres parfaitement synchronisés :

```
const sentences = safeText.match(/[^.!?;:]+[.!?;:]?/g);
const audios = await this.generateAudiosConcurrent(sentences, voice);
return { sentences, audios };  // un MP3 par phrase
```

Le texte est nettoyé du Markdown (gras, italique, code) avant d'être prononcé, afin d'éviter de lire des symboles parasites.

4.8. La génération de l'avatar parlant (SadTalker)

Le service sadtalker.service.js orchestre deux étapes : la synthèse vocale du texte via Edge TTS, puis l'animation de la photo source par le serveur SadTalker exposé via ngrok.

```
async generateAvatarWithText(avatar, text) {
  const audioUrl = await ttsService.synthesizeSpeech(text);   // 1. Edge TTS
  const audioPath = resolve(audioUrl);
  return await this.generateAvatar(avatar, audioPath);        // 2. SadTalker
}
```

Le notebook colab-scripts/SadTalker_API.ipynb télécharge le dépôt officiel OpenTalker/SadTalker, installe les dépendances sur GPU, puis expose une API locale (/generate et /health) rendue accessible depuis Internet grâce à ngrok. Le backend transmet (image + audio) et récupère une vidéo MP4 qu'il stocke dans /uploads/videos.

4.9. Le frontend Angular

Le frontend offre une expérience utilisateur complète : connexion, tableau de bord, consultation des cours, lecture audio avec sous-titres, chat textuel et vocal, création d'avatars et administration. Le service d'authentification stocke le JWT et l'intercepte pour l'ajouter à chaque requête. Les guards protègent les routes privées (AuthGuard) et administrateur (AdminGuard).

L'interface s'appuie sur Tailwind CSS pour la mise en forme et sur un sous-ensemble d'Angular Material (boutons, icônes, menus) pour les composants interactifs.

4.10. Conteneurisation et déploiement

Trois services Docker sont définis dans docker-compose.yml : la base de données, le backend et le frontend. Le backend utilise un build multi-étapes pour optimiser la taille de l'image ; le frontend est compilé puis servi par nginx, qui redirige /api et /uploads vers le backend.

```
services:
  db:       { image: postgres:16-alpine, ... }
  backend:  { build: ./backend,  ports: ['3000:3000'], depends_on: [db] }
  frontend: { build: ./frontend, ports: ['8080:80'],   depends_on: [backend] }
```

En développement, on peut également lancer séparément le backend (node src/app.js) et le frontend (ng serve) avec un proxy, ce qui accélère les itérations.

4.11. Aperçu des interfaces

Cette section présente les principales interfaces de la plateforme, telles qu'elles apparaissent à l'utilisateur final.

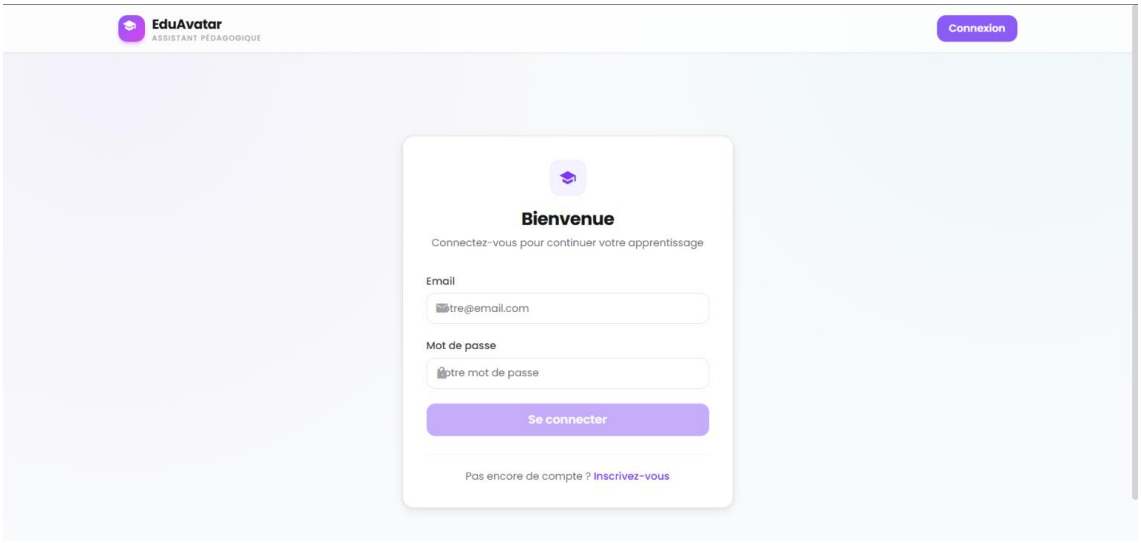


Figure 8 — Page de connexion

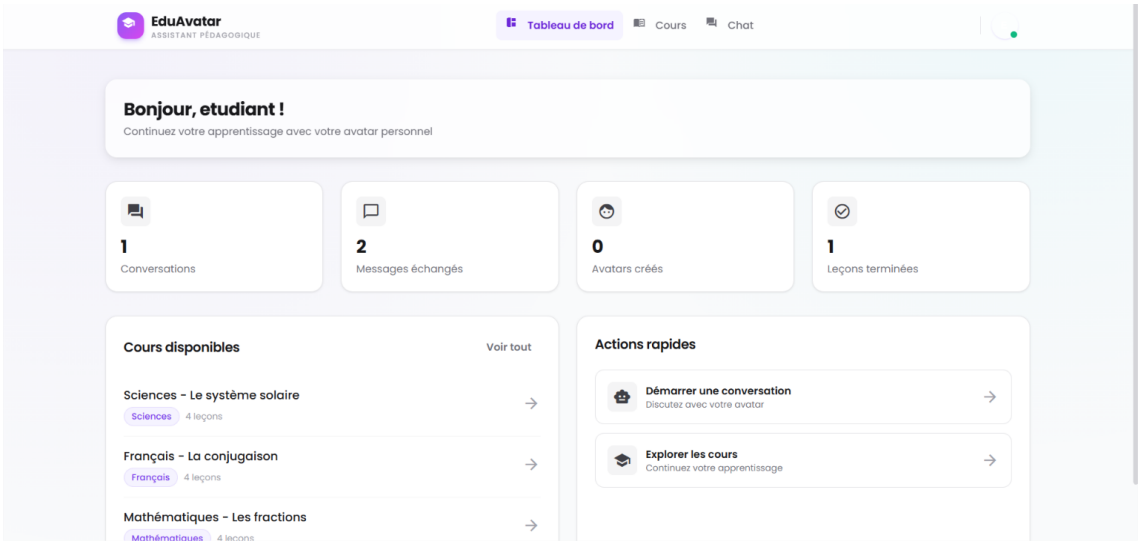


Figure 9 — Tableau de bord de l'apprenant

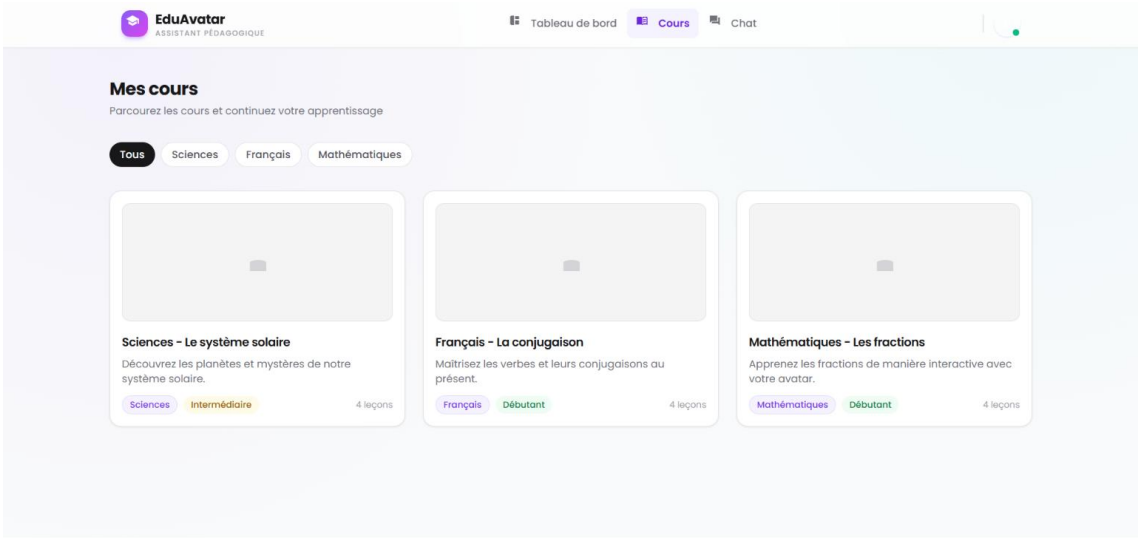


Figure 10 — Liste des cours (apprenant)

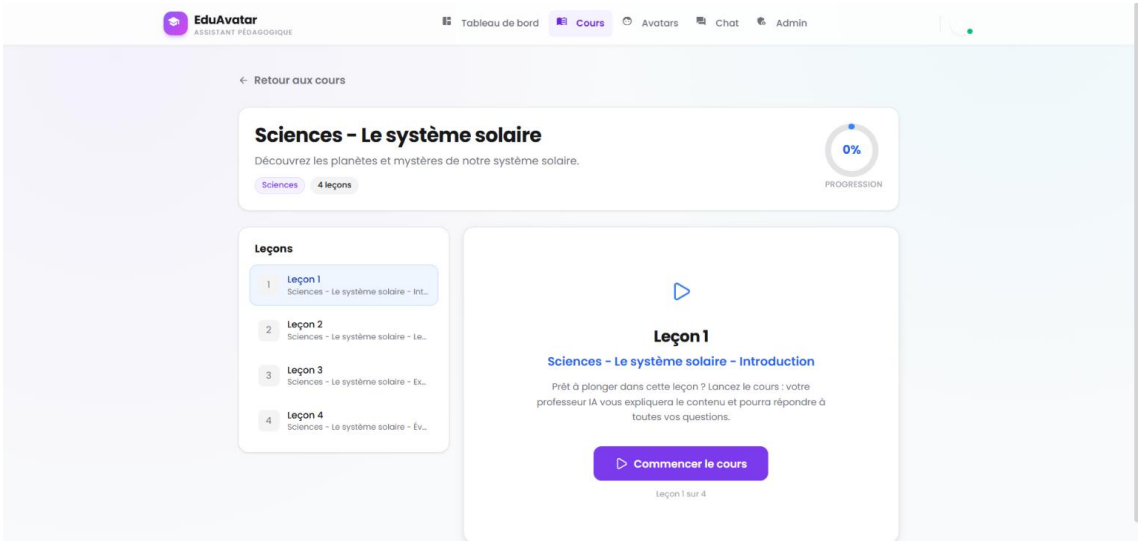


Figure 11 — Détail d'un cours et de ses leçons

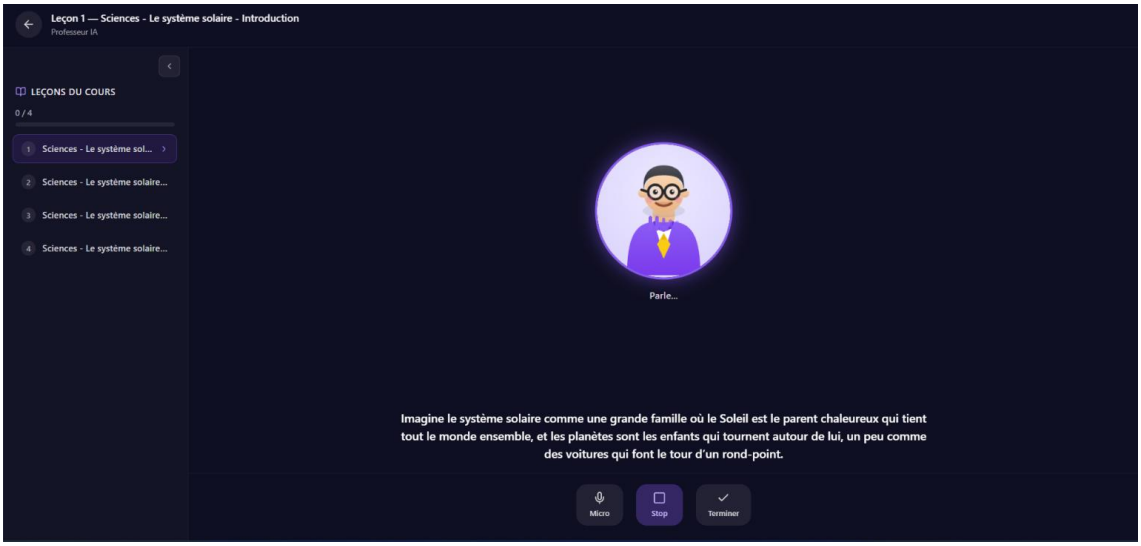


Figure 12 — Explication d'une leçon par l'IA

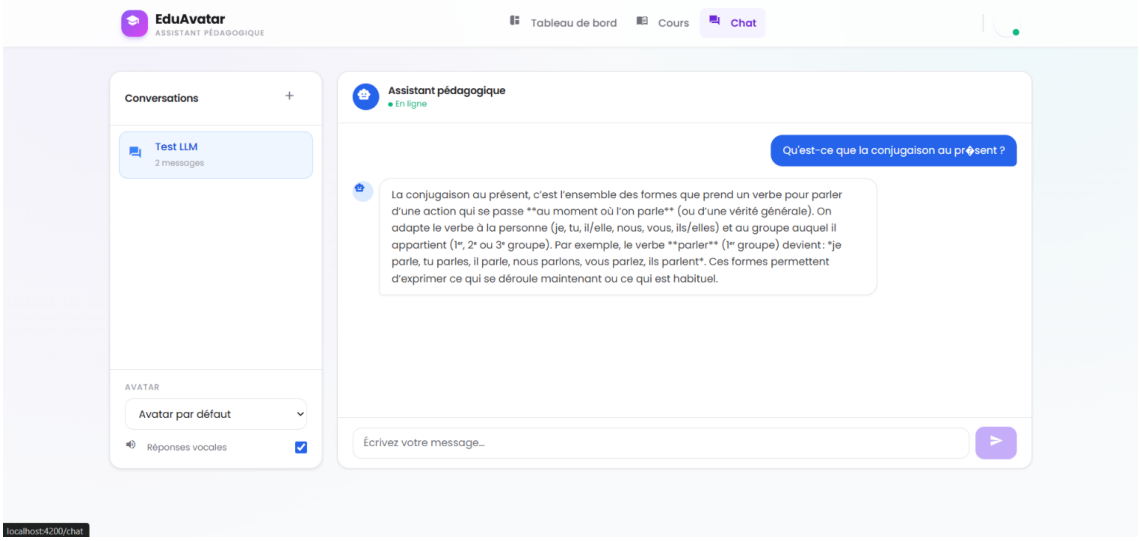


Figure 13 — Chat avec l'avatar pédagogique

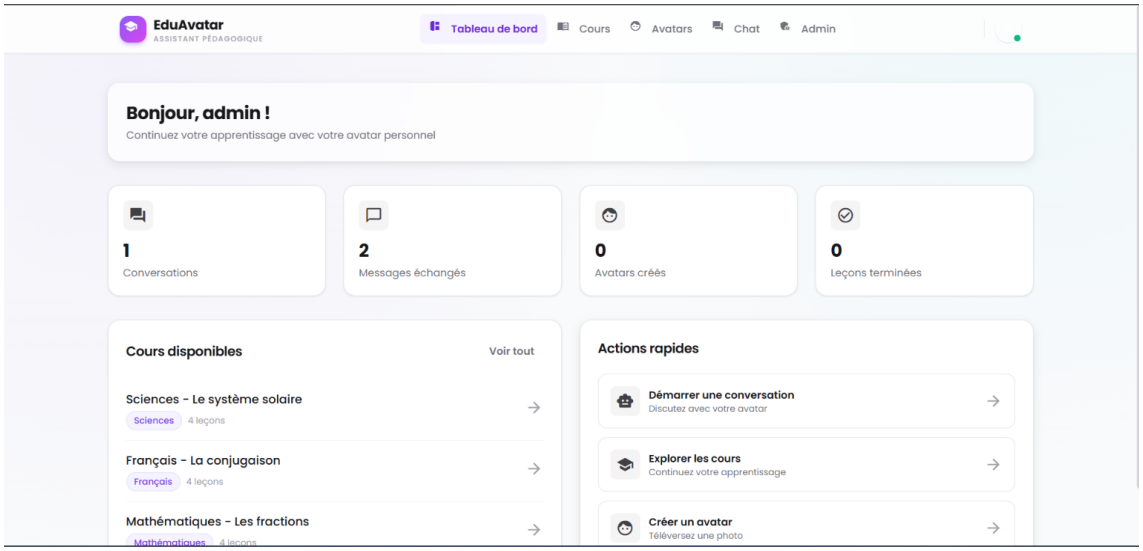


Figure 14 — Tableau de bord administrateur

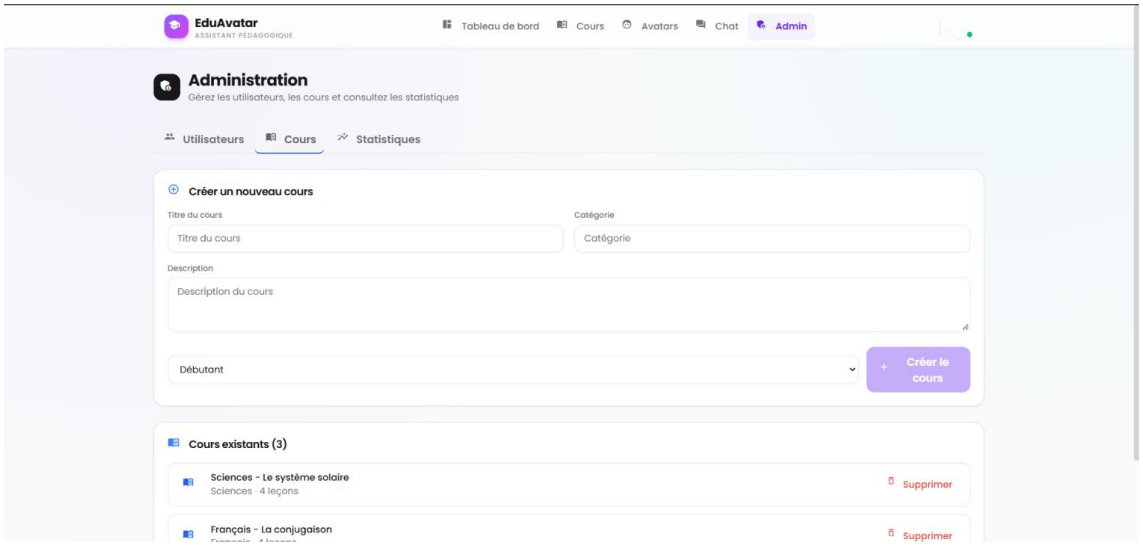


Figure 15 — Ajout d'un nouveau cours

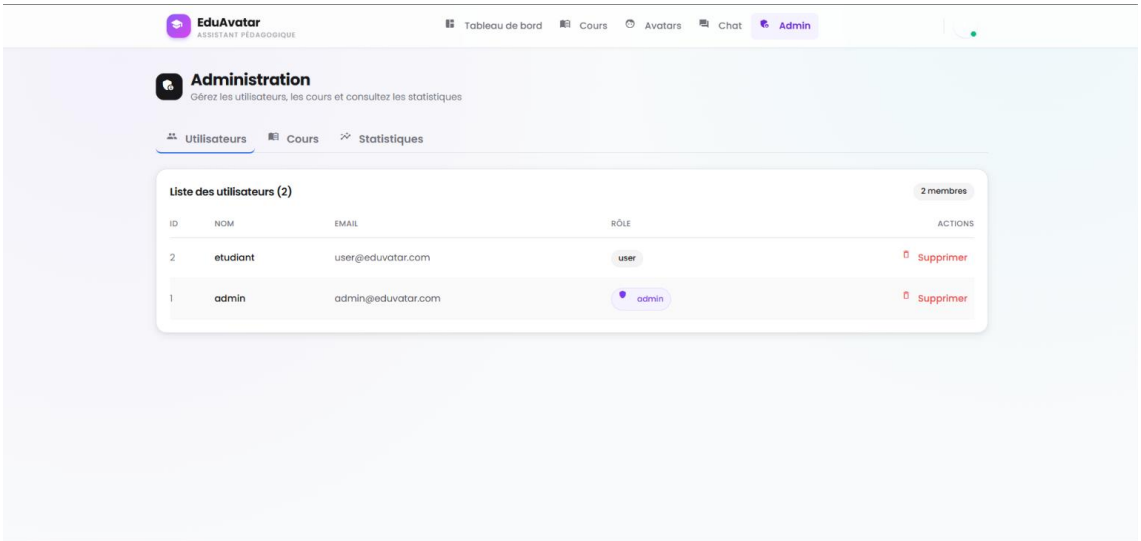


Figure 16 — Gestion des utilisateurs

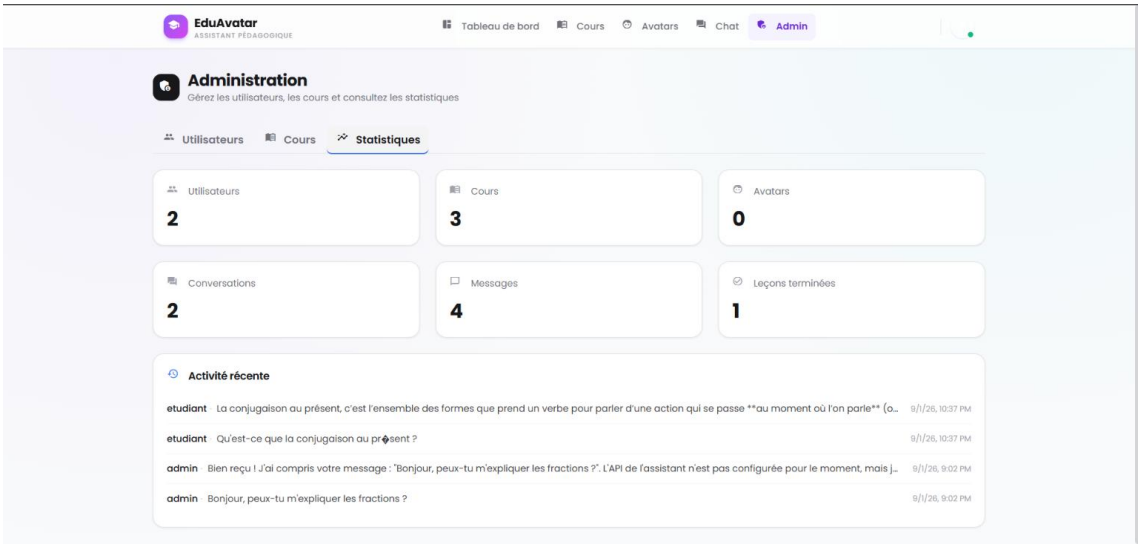


Figure 17 — Page des statistiques

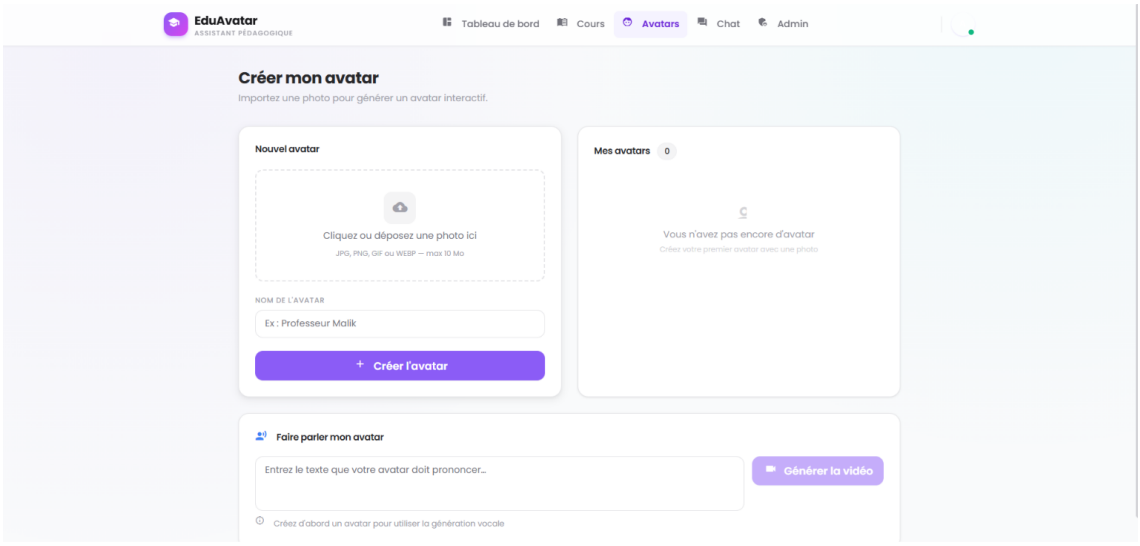


Figure 18 — Création et génération d'un avatar parlant

Chapitre 5. Tests et validation

5.1. Stratégie de test

La validation a porté sur trois niveaux : les tests unitaires des fonctions sensibles, les tests d'intégration de l'API REST, et les tests fonctionnels de bout en bout depuis l'interface utilisateur.

5.2. Scénarios testés

N°	Scénario	Résultat attendu	Statut
T1	Inscription + connexion	JWT émis, accès autorisé	OK
T2	Accès sans token à une route protégée	Erreur 401	OK
T3	Accès admin avec un compte user	Erreur 403	OK
T4	Création d'un cours (admin)	Cours inséré en base	OK
T5	Explication IA d'une leçon	Texte pédagogique renvoyé	OK
T6	Question vocale	Transcription correcte puis réponse	OK
T7	Synthèse vocale par phrases	MP3 générés + sous-titres	OK
T8	Génération d'avatar parlant	Vidéo MP4 produite	OK
T9	Upload d'une image invalide	Erreur 400	OK
T10	Repli LLM indisponible	Réponse de secours renvoyée	OK

Table 10 — Scénarios testés

5.3. Tests de robustesse

Une attention particulière a été portée au comportement du système face aux services externes. Pour chaque brique IA, un mécanisme de repli a été prévu et vérifié :

- LLM : génération d'une réponse par défaut si Groq est indisponible.
- ASR : bascule automatique de Groq vers Hugging Face en cas d'échec.
- TTS : essai successif de plusieurs voix françaises en cas d'échec.
- Avatar : message d'erreur explicite si ngrok/Colab n'est pas joignable.

5.4. Performances observées

Opération	Latence approximative	Remarque
Connexion	< 1 s	Hachage bcrypt + émission JWT
Explication LLM	1 – 3 s	API Groq (inférence rapide)
Transcription vocale	2 – 5 s	Selon la durée de l'audio
Synthèse vocale (par phrase)	1 – 3 s	Génération parallèle (3)

Génération avatar (SadTalker)	1 – 5 min	Dépend du GPU Colab

Table 11 — Performances observées

Ces mesures confirment la viabilité de l'approche pour un usage pédagogique, la seule opération « lourde » restant la génération vidéo, intrinsèquement liée à l'inférence GPU.

Chapitre 6. Bilan, difficultés et perspectives

6.1. Bilan

Le projet EduAvatar a permis de mettre en œuvre une plateforme pédagogique fonctionnelle intégrant toute la chaîne de l'intelligence artificielle conversationnelle et de l'animation d'avatar. Les objectifs fixés ont été atteints : gestion des cours, assistant LLM, reconnaissance vocale, synthèse vocale et génération d'avatar parlant, le tout dans une application web sécurisée et conteneurisée.

6.2. Difficultés rencontrées

- Disponibilité des services gratuits : quotas, latence et nécessité de mécanismes de repli.
- Synchronisation labiale : la génération SadTalker dépend d'un GPU distant (Colab).
- Gestion du réseau : ngrok et certains domaines peuvent être filtrés selon les environnements.
- Synchronisation audio/sous-titres : résolue par la synthèse phrase par phrase.
- Nettoyage du code et gestion des états de chargement dans l'interface.

6.3. Perspectives d'amélioration

- Internationalisation (support multilingue de l'interface et des voix).
- Mise en cache des résultats de synthèse et de transcription pour réduire les appels.
- Ajout d'un mode hors-ligne / service auto-hébergé (Ollama, modèles locaux).
- Déploiement sur un serveur dédié avec HTTPS et CI/CD.
- Tableau de bord analytique plus riche pour les enseignants.
- Tests automatisés (unitaires et end-to-end) intégrés à la chaîne d'intégration continue.

Conclusion générale

Ce projet de fin d'année avait pour ambition de concevoir un système de génération d'avatars parlants au service de l'interaction pédagogique. Le résultat, EduAvatar, démontre qu'il est possible de réunir, à coût nul, une plateforme complète combinant gestion de contenu, intelligence artificielle conversationnelle, reconnaissance et synthèse vocales, et animation faciale.

Sur le plan technique, une architecture trois-tiers claire, une API REST sécurisée et une intégration judicieuse de services gratuits ont permis d'atteindre les objectifs fixés. Sur le plan méthodologique, la démarche a suivi les étapes classiques d'un projet logiciel : analyse des besoins, conception, implémentation, tests et validation.

De nombreuses perspectives restent ouvertes, notamment l'amélioration du photoréalisme et de la latence de l'avatar, l'internationalisation et le déploiement en production. Ce travail constitue ainsi une base solide pour de futurs développements dans le domaine des environnements d'apprentissage intelligents.

Webographie et références

1. Documentation officielle Angular — https://angular.dev
2. Documentation officielle Node.js — https://nodejs.org/docs
3. Documentation Express — https://expressjs.com
4. PostgreSQL Documentation — https://www.postgresql.org/docs
5. Groq API Documentation — https://console.groq.com/docs
6. OpenAI Whisper — https://github.com/openai/whisper
7. Microsoft Edge TTS (msedge-tts) — https://www.npmjs.com/package/msedge-tts
8. SadTalker (OpenTalker) — https://github.com/OpenTalker/SadTalker
9. Hugging Face Inference API — https://huggingface.co/docs/api-inference
10. Docker Documentation — https://docs.docker.com
11. Tailwind CSS — https://tailwindcss.com/docs

Annexes

Annexe A. Variables d'environnement

Variable	Description
POSTGRES_USER / PASSWORD / DB	Identifiants de la base PostgreSQL
JWT_SECRET	Clé secrète de signature des jetons JWT
GROQ_API_KEY	Clé API Groq (LLM + Whisper)
GROQ_MODEL	Modèle LLM utilisé (ex. llama-3.3-70b-versatile)
HF_API_KEY	Clé Hugging Face (repli Whisper)
EDGE_TTS_VOICE	Voix française (fr-FR-DeniseNeural)
SADTALKER_API_URL	URL ngrok du serveur Colab SadTalker
CORS_ORIGIN	Origine autorisée pour le frontend

Table 12 — Variables d'environnement

Annexe B. Comptes de démonstration (seed)

Rôle	Email	Mot de passe
Administrateur	admin@eduvatar.com	admin123
Apprenant	user@eduvatar.com	user123

Table 13 — Comptes de démonstration

Ces comptes sont créés par le script backend/src/seed.js à des fins de démonstration et doivent être modifiés avant tout déploiement réel.

Annexe C. Structure du projet

```
eduvatar-pfa/
├── backend/
│   ├── src/
│   │   ├── config/           # connexion PostgreSQL
│   │   ├── controllers/      # logique des requêtes
│   │   ├── middleware/       # auth JWT, uploads
│   │   ├── models/           # accès aux données
│   │   ├── routes/           # routes REST
│   │   ├── services/         # groq, whisper, tts, sadtalker
│   │   ├── app.js            # point d'entrée
│   │   ├── initDb.js         # création des tables
│   │   └── seed.js           # données de démo
│   └── Dockerfile
├── frontend/
│   ├── src/app/              # composants, services, guards
│   ├── nginx.conf
│   └── Dockerfile
├── colab-scripts/            # notebook SadTalker + ngrok
```

```
├── docker-compose.yml
└── README.md
```